

# An approach for streaming ASR using Bayesian Estimates and speculative processing

# Graph Based User Journey Tracking for Voice AI

## Overview

This document explains a graph based customer journey tracking system for conversational and voice AI systems.

The system uses Bayesian probability updates to estimate:

- Current conversation stage
- Likely next stage
- Stability of the conversation
- Readiness for speculative execution

The design is inspired by NVIDIA speculative speech processing concepts.

Reference:

- NVIDIA Speculative Speech Processing  
[https://github.com/NVIDIA/voice-agent-examples/blob/main/docs/SPECULATIVE\\_SPEECH\\_PROCESSING.md](https://github.com/NVIDIA/voice-agent-examples/blob/main/docs/SPECULATIVE_SPEECH_PROCESSING.md)

---

# Problem Statement

Traditional conversational systems often use fixed intent classification.

Example:

```
```text
User said X -> Intent Y
```

This creates several problems:

- No uncertainty handling
- No gradual confidence updates
- Poor support for streaming voice systems
- Difficult speculative execution
- Hard to model evolving user journeys

A probabilistic graph based system solves these issues.

---

## Core Idea

Treat the conversation as a continuously evolving graph.

Each user utterance updates the probability of possible conversation states.

The system becomes more confident as additional evidence arrives.

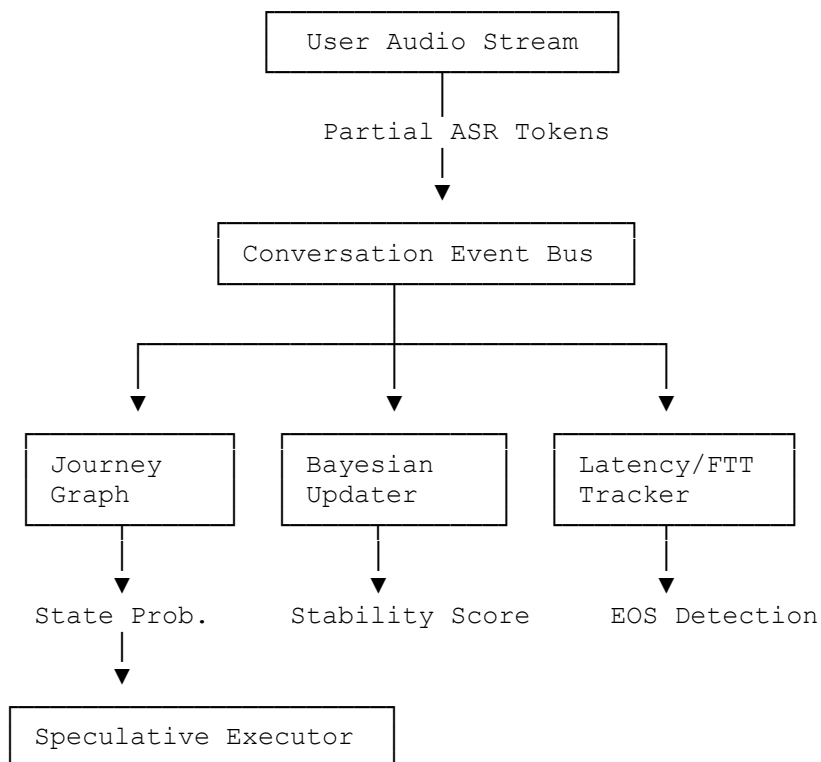
# An approach for streaming ASR using Bayesian Estimates and speculative processing

Example:

```
Audio Stream
↓
ASR Partial Tokens
↓
Bayesian State Updates
↓
Journey Probability Graph
↓
Stability Score
↓
Speculative Execution Decisions
```

---

## High Level Architecture



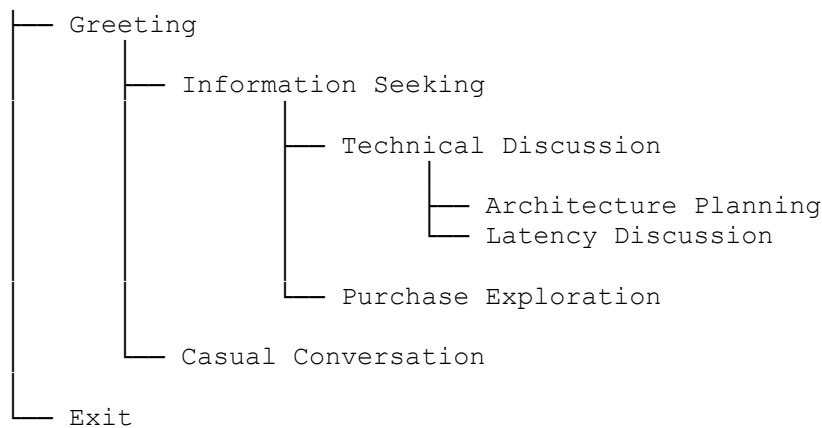
## Journey Graph Representation

Conversation stages are represented as graph nodes.

Example:

```
START
|
```

# An approach for streaming ASR using Bayesian Estimates and speculative processing



Each edge contains:

- Prior probability
- Transition probability
- Evidence weight
- Temporal reinforcement

---

## Bayesian Probability Updates

The system updates probabilities after every user event.

Bayes theorem:

$$P(S_t | E_t) = (P(E_t | S_t) \times P(S_t)) / P(E_t)$$

Where:

- $S_t$  = current state
- $E_t$  = observed evidence

Definitions:

- $P(S_t | E_t)$  = posterior probability
- $P(S_t)$  = prior probability
- $P(E_t | S_t)$  = likelihood
- $P(E_t)$  = marginal probability

---

## Example Conversation Update

# An approach for streaming ASR using Bayesian Estimates and speculative processing

Initial priors:

| State                | Probability |
|----------------------|-------------|
| Greeting             | 0.35        |
| Technical Discussion | 0.25        |
| Sales Inquiry        | 0.15        |
| Support Request      | 0.25        |

User says:

Latency cannot be calculated reliably in stateless mode

Likelihoods:

| Evidence Given State        | Probability |
|-----------------------------|-------------|
| P(latency talk   Technical) | 0.9         |
| P(latency talk   Sales)     | 0.1         |
| P(latency talk   Support)   | 0.4         |

Updated probabilities:

| State                | Posterior |
|----------------------|-----------|
| Technical Discussion | 0.71      |
| Support              | 0.18      |
| Greeting             | 0.07      |
| Sales                | 0.04      |

---

## Stability Score

The system should estimate how stable the current hypothesis is.

Higher stability means:

- Conversation direction is clear
  - Speculative execution becomes safer
  - Backend calls can start earlier
- 

## Entropy Based Stability

Entropy measures uncertainty.

# An approach for streaming ASR using Bayesian Estimates and speculative processing

Formula:

$$H(S) = -\sum P(s_i) \log P(s_i)$$

Lower entropy means higher confidence.

Example:

| Stage                     | Stability |
|---------------------------|-----------|
| Early conversation        | 0.32      |
| Mid discussion            | 0.68      |
| Deep technical discussion | 0.91      |

---

## Practical Stability Formula

Simple implementation:

$$Stability = \alpha \times P_{top} + \beta \times Reinforcement - \gamma \times Entropy$$

Where:

- $P_{top}$  = highest state probability
  - $Reinforcement$  = repeated evidence count
  - $Entropy$  = uncertainty score
- 

## Speculative Execution Thresholds

| Stability Score | Action                      |
|-----------------|-----------------------------|
| < 0.4           | No speculation              |
| 0.4 - 0.7       | Prepare candidate responses |
| > 0.7           | Start backend calls         |
| > 0.9           | Pre-render responses        |

---

## Customer Journey Pallets

The conversation graph can be layered.

Example:

# An approach for streaming ASR using Bayesian Estimates and speculative processing

Layer 1 -> Conversation Type  
Layer 2 -> User Goal  
Layer 3 -> Technical Depth  
Layer 4 -> Purchase Readiness

---

## Suggested Implementation Strategy

### Phase 1

Use simple rule based updates with Bayesian normalization.

Example:

```
if "latency" in utterance:
    likelihood["technical"] += 0.3

if "implementation" in utterance:
    likelihood["architecture"] += 0.4
```

Goals:

- Fast implementation
  - Easy debugging
  - Minimal complexity
- 

### Phase 2

Add temporal reinforcement.

Signals:

- Repeated topics
- Topic persistence
- Conversation duration
- Consecutive confirmations

Example:

```
3 consecutive technical turns
→ increase confidence multiplier
```

---

### Phase 3

# An approach for streaming ASR using Bayesian Estimates and speculative processing

Move to probabilistic graph models.

Possible approaches:

- Bayesian Networks
- Bayesian Markov Chains
- Hidden Markov Models

Possible graph nodes:

| Node Type           | Example              |
|---------------------|----------------------|
| Intent              | Technical Discussion |
| Sophistication      | Advanced             |
| Readiness           | Medium               |
| Urgency             | High                 |
| Latency Sensitivity | High                 |

---

## Voice AI Specific Signals

Additional voice signals can improve accuracy.

| Signal         | Interpretation            |
|----------------|---------------------------|
| Interruptions  | Confusion or urgency      |
| Long pauses    | Cognitive load            |
| Faster speech  | Excitement or urgency     |
| Repetition     | Low confidence            |
| EOS confidence | Turn completion certainty |

---

## First Token Time (FTT)

FTT calculation requires end of speech detection.

Important insight:

Latency measurement starts only after EOS detection.

Speech states:

```
speaking
possible_eos
confirmed_eos
```

# An approach for streaming ASR using Bayesian Estimates and speculative processing

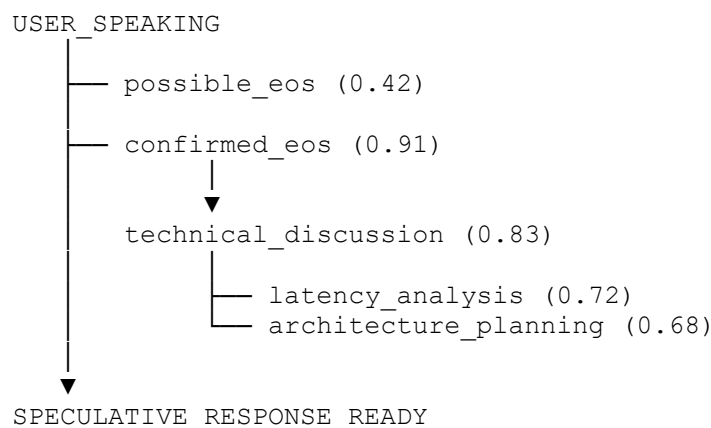
The system should enter:

LLM\_RESPONSE\_PENDING

only after EOS confidence passes a threshold.

---

## Unified Conversation State Example



## Recommended Data Model

```
class JourneyNode:
    id: str
    prior: float
    posterior: float
    stability: float
    evidence_count: int

class JourneyEdge:
    source: str
    target: str
    transition_prob: float
```

---

## Why This Works Well

Advantages:

1. Dynamic probability updates
2. Better uncertainty handling
3. Supports speculative execution
4. Works with streaming voice input
5. Handles evolving conversations naturally



# An approach for streaming ASR using Bayesian Estimates and speculative processing

6. Easier integration with latency optimization

---

## Relation to NVIDIA Speculative Speech Processing

The NVIDIA speculative approach focuses on:

```
confidence accumulation  
→ early execution  
→ rollback if needed
```

The Bayesian journey graph adds:

- conversation trajectory estimation
- stability scoring
- probabilistic orchestration
- adaptive speculative execution

This enables:

```
conversation level speculative execution
```

instead of only token level speculation.

---

## References

### NVIDIA Speculative Speech Processing

[https://github.com/NVIDIA/voice-agent-examples/blob/main/docs/SPECULATIVE\\_SPEECH\\_PROCESSING.md](https://github.com/NVIDIA/voice-agent-examples/blob/main/docs/SPECULATIVE_SPEECH_PROCESSING.md)

### Bayesian Customer Journey Modeling

<https://insights.axtria.com/hubfs/thought-leadership-whitepapers/Axtria-Insights-WP-Data-Driven-Customer-Journey-Design-Using-Bayesian-Network-Methodology.pdf>